

完整软件过程定义文档

项目名称：校园二手交易平台

学生：文就南

学号：2023141461219

过程版本：V2.0（AI 增强版）

1. 过程概述

校园二手交易平台面向校内学生，提供二手商品发布、搜索浏览、收藏沟通、订单交易、评价举报和后台审核等功能。项目采用“迭代式生命周期 + 阶段质量门禁 + AI 辅助工程活动”的软件过程。过程目标是在保证需求可追踪、设计可评审、代码可维护、测试可度量、运维可复盘的基础上，引入 AI 技术提升需求、设计、编码、测试和运维阶段的效率与质量。

本过程强调人机协同。AI 可参与初稿生成、候选方案生成、检查补全、用例扩展、日志摘要和风险提示，但不能独立作出需求确认、架构决策、代码合并、测试通过和生产发布决定。所有 AI 输出必须经过对应负责人评审，重要输出纳入配置管理。

2. 生命周期模型

项目采用四周一轮的短迭代模式。每轮迭代包含需求分析、设计、实现、测试、部署与复盘。课程项目可设置两个主要迭代：第一迭代完成用户认证、商品发布、商品浏览；第二迭代完成搜索、订单、评价举报和后台审核。每轮迭代结束必须形成可演示版本、测试报告和过程度量记录。

flowchart LR

A[迭代启动] --> B[需求分析]

B --> C[设计建模]

C --> D[编码实现]

D --> E[测试验证]

E --> F[部署演示]

F --> G[复盘改进]

G --> A

3. 角色职责

角色	主要职责	关键交付物
----	------	-------

---	---	---
-----	-----	-----

项目经理	制定计划、协调进度、组织会议、跟踪风险	项目计划、会议纪要、风险清单
------	---------------------	----------------

需求负责人	组织访谈、维护需求规格、确认验收标准	需求规格说明书、需求追踪矩阵
-------	--------------------	----------------

架构/设计负责人	设计系统结构、接口、数据库和关键机制	架构设计文档、接口说明、数据库设计
----------	--------------------	-------------------

开发负责人	分配开发任务、维护代码规范、组织代码审查	代码仓库、提交记录、审查记录
-------	----------------------	----------------

测试负责人	制定测试计划、设计用例、管理缺陷	测试计划、测试用例、缺陷报告
-------	------------------	----------------

运维负责人	部署环境、监控指标、处理故障	部署说明、监控记录、故障复盘
-------	----------------	----------------

AI 协同负责人	维护 AI 使用规范、提示词模板和输出归档	AI 使用记录、提示词库、采纳率统计
----------	-----------------------	--------------------

数据安全负责人	审查敏感数据、脱敏材料和合规风险	脱敏记录、安全检查表
---------	------------------	------------

4. 需求分析过程

需求阶段目标是明确用户目标、业务边界、功能需求、非功能需求和验收标准。输入包括课程要求、用户访谈、同类平台分析和团队讨论记录。活动步骤如下：

1. 项目经理组织迭代启动会，确定本轮迭代范围。
2. 需求负责人收集用户需求，形成访谈纪要和功能列表。
3. AI 需求分析员根据脱敏材料生成用户故事、验收标准、异常场景和待确认问题。
4. 需求负责人人工筛选 AI 输出，删除不真实需求，补充业务规则。
5. 团队召开需求评审会，检查完整性、一致性、可测性和优先级。
6. 评审通过后形成需求规格说明书和需求追踪矩阵。

需求质量标准包括：每个核心功能至少要有用户故事、主成功场景、异常场景、验收标准；每条需求必须有唯一编号；需求变更必须记录原因、影响范围和批准人；AI 生成内容必须标注“采纳、修改、废弃”状态。

5. 系统设计过程

设计阶段目标是把需求转化为可实现的架构、模块、接口和数据结构。校园二手交易平台采用前后端分离架构，后端提供 REST API，数据库保存用户、商品、订单、评价、举报和审核记录。设计活动如下：

1. 架构负责人根据需求划分模块，包括用户模块、商品模块、搜索模块、订单模块、消息模块、评价举报模块、后台审核模块。
2. AI 辅助生成架构备选方案、接口草案、数据库字段建议和风险清单。
3. 架构负责人结合课程规模和团队能力选择最终方案。
4. 设计评审检查模块边界、接口一致性、数据库约束、安全策略和异常处理。
5. 评审通过后冻结迭代设计基线，后续变更需记录。

```
flowchart TD
    U[用户端] --> API[后端API服务]
    A[管理员端] --> API
    API --> AUTH[认证与权限模块]
    API --> GOODS[商品模块]
    API --> ORDER[订单模块]
    API --> REVIEW[评价举报模块]
    API --> SEARCH[搜索模块]
    API --> DB[(数据库)]
    API --> OSS[图片存储]
    API --> LOG[日志与监控]
```

设计质量标准包括：接口必须写明请求参数、响应字段、错误码和权限要求；数据库表必须说明主键、外键、唯一约束和索引；关键流程必须有状态机或流程图；AI 生成图表必须经人工校对，不得出现不存在的模块或接口。

6. 编码实现过程

编码阶段目标是按设计完成可运行、可维护、可测试的程序。开发活动遵循分支管理和代码审查制度。每个任务从需求编号或缺陷编号创建，开发者在独立分支实现，提交前完成自测和静态检查。

AI 可辅助生成样板代码、DTO、参数校验、单元测试、注释和局部重构建议。开发者使用 AI 时必须遵守三条规则：不得上传密钥、真实用户数据和未脱敏日志；不得直接提交无法解释的 AI 代码；涉及认证、权限、交易状态和数据删除的代码必须人工逐行审查。

代码质量标准包括：命名清晰、函数职责单一、异常处理完整、日志不泄露隐私、接口返回格式统一、核心业务有单元测试。代码合并前必须通过开发者自查、AI 辅助审查、同伴审查和自动化测试。

7. 测试验证过程

测试阶段目标是验证软件满足需求并发现缺陷。测试范围包括功能测试、接口测试、边界值测试、权限测试、安全测试、兼容性测试和回归测试。活动步骤如下：

1. 测试负责人根据需求追踪矩阵制定测试计划。
2. AI 测试分析师根据需求和接口文档生成候选测试用例。
3. 测试负责人对 AI 用例进行去重、补充、优先级排序。
4. 开发者和测试负责人共同确认高风险模块。
5. 执行测试并记录结果，发现缺陷后提交缺陷报告。
6. 缺陷修复后执行回归测试。
7. 形成测试报告和质量评估。

测试质量标准包括：P0/P1 需求必须全部有测试用例；核心接口必须有正常、异常、边界和权限场景；严重缺陷必须关闭后才能发布；AI 生成用例的有效率和缺陷发现率纳入过程度量。

8. 部署与运维过程

部署阶段目标是把通过测试的版本发布到演示或生产环境，并保证可监控、可回滚。部署前检查配置文件、数据库迁移脚本、依赖版本、环境变量和初始数据。部署后观察关键指标，包括接口错误率、平均响应时间、登录失败率、图片上传失败率、订单异常数和后台审核积压数。

AI 可用于日志摘要和根因建议。运维负责人输入脱敏日志、指标窗口和最近变更记录，AI 输出可能原因、影响范围和排查步骤。运维负责人确认后执行处理。所有故障必须复盘，记录故障时间、影响、根因、修复措施和预防措施。

9. 配置管理与文档管理

项目采用 Git 进行版本管理。主分支保存稳定版本，开发分支保存迭代开发内容，功能分支对应具体任务。提交信息应包含需求编号或缺陷编号。文档包括需求规格说明书、设计文档、接口文档、测试用例、测试报告、部署说明、风险清单和 AI 使用记录。所有文档采用 Markdown 编写，重要图示使用 Mermaid。

AI 输出也属于项目资产。提示词、输入摘要、输出结果、人工修改说明和采纳状态应保存到 AI 使用记录中。这样既便于复盘，也能避免“AI 说过但没人负责”的过程风险。

10. 质量保证与评审机制

本过程设置四个质量门禁：

门禁	进入条件	检查内容	通过标准
需求评审	需求初稿完成	完整性、一致性、可测性、优先级	无 P0/P1 未确认问题
设计评审	架构和接口草案完成	模块边界、接口、数据库、安全	关键风险有处理方案
代码评审	功能开发完成	规范、安全、可读性、测试	无阻塞问题，测试通过
发布评审	测试完成	缺陷状态、部署准备、回滚方案	严重缺陷关闭，可回滚

AI 辅助检查可以提高发现问题的概率，但不替代评审会议。评审记录必须写明问题、责任人、截止时间和关闭状态。

11. 度量指标

过程度量用于判断改进是否有效。效率指标包括需求初稿耗时、设计文档编写耗时、代码实现耗时、测试用例设计耗时、缺陷定位耗时。质量指标包括需求评审缺陷数、设计评审问题数、代码审查问题数、测试通过率、缺陷密度、缺陷逃逸率。AI 指标包括 AI 输出采纳率、AI 输出修改率、AI 生成用例有效率、AI 发现问题数、AI 幻觉纠正数。风险指标包括敏感数据拦截次数、安全问题数和高风险 AI 建议拒绝次数。

建议每轮迭代结束生成过程度量表。例如：需求初稿耗时从 5 小时降至 3 小时；测试用例设计耗时从 45 分钟降至 26 分钟；AI 生成用例有效率达到 85%；AI 输出中需人工纠正的问题 7 项。只有同时观察效率和质量，才能避免“速度变快但缺陷更多”的假改进。

12. 风险管理

项目风险包括需求变更频繁、成员时间冲突、接口联调延期、测试覆盖不足、部署环境不稳定、AI 输出幻觉、敏感数据泄露和过度依赖 AI。风险管理流程为识别、评估、应对、跟踪和关闭。高风险项必须指定责任人和应对措施。

对于 AI 特有风险，过程规定如下：禁止向外部模型输入真实手机号、密码、密钥、聊天记录和未脱敏日志；AI 生成的需求、设计和代码必须人工确认；AI 生成的参考资料必须核验来源；AI 建议如影响安全、权限、交易和数据一致性，必须由负责人二次审查。

13. 交付物清单

阶段	交付物	负责人	质量要求
需求	需求规格说明书、用户故事、验收标准、追踪矩阵	需求负责人	编号完整，可测试
设计	架构图、接口文档、数据库设计、状态图	设计负责人	约束清晰，评审通过
编码	源代码、单元测试、代码审查记录	开发负责人	测试通过，无阻塞问题
测试	测试计划、测试用例、缺陷报告、测试总结	测试负责人	覆盖 P0/P1 需求
运维	部署说明、监控记录、故障复盘	运维负责人	可回滚，可追踪
AI 管理	提示词库、AI 输出记录、采纳率统计	AI 协同负责人	输入脱敏，输出可追溯

14. 过程改进机制

每轮迭代结束召开复盘会，围绕四个问题讨论：哪些活动耗时超出预期，哪些缺陷本可以提前发现，哪些 AI 输出真正有价值，哪些 AI 输出造成返工。复盘结果进入下一轮过程改进。若某个 AI 场景连续两轮采纳率低于 50% 或纠错成本高于节省时间，应暂停该场景并重新设计提示词或工具链。

15. 总结

本过程定义文档将传统软件过程与 AI 技术结合，形成适合校园二手交易平台的 AI 增强型软件过程。它保留需求评审、设计评审、代码审查、测试报告和发布门禁等软件工程基本纪律，同时把 AI 固定到需求建模、方案比较、代码辅助、测试生成和日志分析等高质量节点。通过角色职责、交付物、质量标准、风险控制和度量指标的系统定义，AI 不再是临时使用的个人工具，而成为可管理、可追踪、可评估的过程能力。

16. 需求变更管理

需求变更是课程项目和真实项目中都很常见的问题。任何需求变更必须通过变更申请、影响分析、审批、实施和验证五个步骤。变更申请需说明变更原因、涉及模块、期望上线时间和优先级；影响分析需评估对需求、设计、代码、测试、部署和文档的影响；审批由项目经理组织，需求负责人、设计负责人、开发负责人和测试负责人共同确认；实施完成后必须更新需求追踪矩阵和测试用例。

AI 可以辅助进行影响分析。例如输入变更描述“商品发布增加议价功能”，AI 可提示该变更可能影响商品详情页、聊天模块、订单价格确认、测试用例和数据库字段。但 AI 只能提供候选影响清单，最终影响范围仍由团队确认。

17. 缺陷管理过程

缺陷管理包括缺陷提交、确认、分派、修复、回归和关闭。缺陷报告必须包含缺陷编号、标题、严重级别、优先级、复现步骤、测试环境、期望结果、实际结果、截图或日志、责任人和状态。严重级别分为致命、严重、一般、轻微；优先级分为 P0、P1、P2、P3。

AI 可用于缺陷摘要、重复缺陷识别和可能原因分析。例如多个图片上传失败缺陷可能都指向文件大小校验或对象存储配置问题。AI 分析结果应写入缺陷备注，但不能直接关闭缺陷。缺陷关闭必须由测试负责人根据回归结果确认。

18. 安全与隐私过程

校园二手交易平台涉及用户账号、联系方式、聊天内容、商品图片和交易记录，必须设置安全与隐私要求。账号密码不得明文存储；接口必须校验登录状态和资源归属；日志不得记录密码、验证码、完整手机号和敏感聊天内容；管理员后台必须区分普通用户和管理员权限；删除和下架操作必须记录审计日志。

AI 使用中的隐私要求同样重要。输入 AI 的材料必须脱敏，真实手机号可替换为 1380000，真实姓名可替换为用户A，密钥和数据库连接串禁止输入。若需要分析日志，应截取必要字段并删除 token、cookie、IP 精确地址和个人信息。数据安全负责人负责抽查 AI 输入记录。

19. AI 使用记录模板

字段	说明
---	---
使用编号	AI-YYYYMMDD-序号
使用阶段	需求/设计/编码/测试/运维
使用目的	例如生成测试用例、检查需求冲突
输入摘要	说明输入内容，不保存敏感信息
使用工具	例如 ChatGPT、Copilot、Codex
输出摘要	说明 AI 主要输出
采纳状态	采纳/修改后采纳/废弃
人工修改点	记录关键修改
风险备注	幻觉、隐私、安全、版权等问题

该模板能够把 AI 使用从个人经验转化为团队过程资产。复盘时可以统计哪些提示词最有效、哪些场景采纳率低、哪些输出容易出错。

20. 需求追踪矩阵模板

需求编号	需求描述	设计项	代码模块	测试用例	状态
---	---	---	---	---	---
REQ-001	用户登录	认证模块设计	auth-service	TC-AUTH-001	已验证
REQ-002	发布商品	商品模块设计	goods-service	TC-PUB-001	已验证
REQ-003	管理员审核	审核流程设计	admin-review	TC-PUB-019	已验证
REQ-004	商品举报	举报模块设计	report-service	TC-RPT-001	待测试

需求追踪矩阵保证每条需求都能追踪到设计、代码和测试，避免出现“做了但没人测”或“测了但需求不存在”的问题。AI 可以辅助检查矩阵缺口，但矩阵维护责任仍属于需求负责人和测试负责人。

21. 发布与回滚清单

发布前必须检查代码版本、数据库脚本、配置文件、依赖版本、测试报告、已知问题、监控指标和回滚方案。若发布后出现严重问题，应按回滚方案恢复到上一稳定版本，并记录故障复盘。课程项目虽然规模较小，也应保留发布清单，以体现规范的软件过程管理。

检查项	是否通过	备注
---	---	---
主分支代码已冻结	是/否	记录提交号
P0/P1 缺陷已关闭	是/否	附测试报告
数据库脚本已备份	是/否	附脚本版本
配置文件已核对	是/否	不含明文密钥
回滚步骤已确认	是/否	指定负责人
监控指标已开启	是/否	错误率、耗时

22. 过程度量表模板

指标	定义	采集方式	目标值
---	---	---	---
需求初稿耗时	从资料整理到初稿完成的时间	人工记录	降低 30%-40%
AI 输出采纳率	采纳或修改后采纳数量/总输出数量	AI 使用记录	高于 70%
测试用例有效率	有效用例/AI 生成候选用例	测试评审记录	高于 80%
缺陷逃逸率	发布后缺陷/总缺陷	缺陷库	越低越好
代码审查问题密度	审查问题数/千行代码	审查记录	持续下降
敏感输入拦截次数	AI 输入前发现的敏感信息次数	安全检查记录	有记录可追踪

这些指标可以帮助团队判断 AI 改进是否真正有效。若 AI 输出采纳率高但缺陷逃逸率上升，说明团队可能过度追求速度；若 AI 输出采纳率低，说明提示词或场景选择需要调整。

23. 复盘会议模板

每轮迭代结束后召开复盘会，会议时间控制在 30-45 分钟。复盘内容包括：本轮完成了哪些需求，哪些活动延迟，哪些缺陷最严重，哪些 AI 辅助最有效，哪些 AI 输出造成返工，下轮应改进什么。会议结论必须形成行动项，明确责任人和完成时间。

复盘不应只写“继续努力”这类空泛结论，而应形成可执行改进。例如“商品发布测试遗漏未登录接口调用，下轮所有接口测试必须包含未登录、无权限和资源归属三类用例”。这种具体行动项才能推动过程成熟。

24. 附录：商品发布模块过程实例

以商品发布模块为例，过程执行如下。需求阶段收集“学生希望发布二手教材、电子产品和生活用品”的用户需求，AI 生成用户故事和验收标准，需求负责人确认标题长度、价格范围、图片数量和审核规则。设计阶段确定商品表、图片表和审核状态字段，AI 提示需要考虑图片大小和敏感词。编码阶段开发者实现发布接口、图片上传和状态保存，AI 辅助生成参数校验和单元测试。测试阶段 AI 生成 34 条候选用例，人工筛选为 29 条有效用例。部署阶段监控图片上传失败率和发布接口错误率。复盘阶段记录 AI 在测试设计中节省约 42% 时间，但仍需人工修订规则。

该实例说明，AI 增强过程不是抽象口号，而是可以落到每个模块、每个交付物和每个质量门禁中。

参考文献与资料

- Cheng H., Husen J. H., Lu Y. 等. Generative AI for Requirements Engineering: A Systematic Literature Review, arXiv, 2024. <https://arxiv.org/abs/2409.06741>
- Huang Y., Chen Y., Chen X. 等. Generative Software Engineering, arXiv, 2024. <https://arxiv.org/abs/2403.02583>
- Pandey R., Singh P., Wei R., Shankar S. Transforming Software Development: Evaluating the Efficiency and Challenges of GitHub Copilot in Real-World Projects, arXiv, 2024. <https://arxiv.org/abs/2406.17910>

4. Song F., Agarwal A., Wen W. The Impact of Generative AI on Collaborative Open-Source Software Development: Evidence from GitHub Copilot, arXiv, 2024. <https://arxiv.org/abs/2410.02091>
5. Li H., Zhang H., Hassan A. E. The Rise of AI Teammates in Software Engineering (SE) 3.0, arXiv, 2025. <https://arxiv.org/abs/2507.15003>
6. OpenAI. Introducing Codex, 2025. <https://openai.com/index/introducing-codex/>
7. GitHub Docs. GitHub Copilot documentation, 2025–2026. <https://docs.github.com/en/copilot>
8. Microsoft. Playwright MCP documentation, 2025–2026. <https://playwright.dev/>